

JUST FOR US

Personal cheat sheet, not a graded submission -- everything about Part 1 and Part 2, in plain language, so any question about the project can be answered on the spot.

Big picture

Two-part face-morphing assignment, one repo, one Python environment. Part 1 builds face morphs -- blending two people's faces into one convincing single image. Part 2 builds a detector that tries to catch those morphs from a single suspect image, with no second trusted photo to compare against.

Folder map

Repo layout:

```
src/
  landmarks.py      Part 1: face landmark detection (MediaPipe)
  morph.py          Part 1: triangulation + warping + blending (the core algorithm)
  img_io.py         shared: unicode-safe image read/write
  app.py            Part 1: interactive Streamlit demo app
  build_showcase.py Part 1: builds the 'A | Morph | B' triptychs for the report
  build_report_pdf.py Part 1: generates submission_report.pdf
  build_ml_sdlc_pdf.py a separate SDLC process-documentation report
  mad_features.py   Part 2: feature extraction (color space, pyramid, LBP/HOG/BSIF)
  mad_crc.py        Part 2: the classifier (CRC) + score fusion
  train_mad.py      Part 2: runs training/evaluation end to end
  build_mad_report_pdf.py Part 2: generates submission_report_part2.pdf

outputs/
  pairs/           Part 1 input: pairXXX_A.png, pairXXX_B.png (original bona fide faces)
  morphs/          Part 1 output: pairXXX_morph.png (blended attack image, alpha=0.5)
  strips/          Part 1 output: one row per pair, alpha = 0 / .25 / .5 / .75 / 1
  showcase/        Part 1 output: 5 labelled triptychs used in the report
  morph_manifest.json list of which pair IDs exist (58 of them)
  mad/             Part 2 output: metrics.json, det_curve.png, scores.npz

models/face_landmarker.task pretrained MediaPipe model Part 1 needs
references/         the Part 2 paper we implemented (PDF)
submission_report.pdf Part 1's 2-page report
submission_report_part2.pdf Part 2's 2-page report
```

Note: the images in outputs/pairs/ come from LFW (Labeled Faces in the Wild). The script that originally downloaded/paired them (fetch_data.py) was deleted in an earlier cleanup commit -- only its output images stayed. So there is no 'download data' script in src/ anymore.

Part 1 -- Face Morphing

Goal: given two people's face photos, produce one image that blends both identities -- a 'morphing attack' image.

The 5 steps, in order

1. Landmark detection (`landmarks.py::get_landmarks`) -- MediaPipe's FaceLandmarker model finds 478 corresponding points on the face (eyes, nose, mouth corners, jawline, etc.) in each of the two photos. 'Corresponding' means point #37 on face A and point #37 on face B are always the same anatomical spot, so the two point-sets are directly comparable.
2. Boundary padding -- 8 extra fixed points (4 corners + 4 edge midpoints of the image) get appended to both point-sets. Without these, only the face itself would warp and the background/hair would smear or shear oddly.
3. Delaunay triangulation (`morph.py::delaunay_triangulation`) -- average the two point-sets $((1-\alpha)*ptsA + \alpha*ptsB)$ into one 'target shape', then triangulate that shape into a mesh of triangles (OpenCV's `Subdiv2D`). One triangle mesh is valid for both source images because both point-sets share the same point ordering.
4. Piecewise affine warp (`morph.py::_warp_triangle`) -- for every triangle, compute the affine transform mapping that triangle's shape in image A onto its shape in the target mesh, and warp just that triangular patch (`cv2.warpAffine`). Do this once using A's triangles and once using B's triangles: two full images, both now reshaped into the same averaged geometry.
5. Cross-dissolve -- blend the two reshaped images pixel by pixel: $morph = (1-\alpha)*warpedA + \alpha*warpedB$. At $\alpha=0.5$ you get an even 50/50 blend of both identities' geometry and texture.

α controls the mix: 0 = pure A, 1 = pure B, 0.5 = the 'attack' image used for the report.

Two ways this pipeline gets used

- Batch/offline: whatever script built `outputs/pairs`, `outputs/morphs`, `outputs/strips` (that generation script itself was removed, only its outputs remain committed) -> feeds `build_showcase.py` (labels 5 chosen pairs as triptychs) -> feeds `build_report_pdf.py` (assembles `submission_report.pdf`).
- Interactive: `app.py`, a Streamlit web app. Upload any two photos live. It auto-detects and crops both faces to a matching scale first (`prepare_pair`), then can composite the blended face only onto one subject's actual photo (keeping their real hair/background) using OpenCV Poisson seamless cloning to smooth the skin-tone seam -- mirroring how a real morphing attack is assembled. Run with `run_app.ps1` (creates an ASCII-only folder junction first, since MediaPipe can't start from this repo's Hebrew-named path).

Part 2 -- Morphing Attack Detection (S-MAD)

Goal: given one single face image (no reference photo to compare against), decide: bona fide or morph? We reimplemented Venkatesh, Raghavendra, Raja & Busch, 'Single Image Face Morphing Attack Detection Using Ensemble of Features', IEEE FUSION 2020 (PDF saved in references/).

Why this type of detection (S-MAD = single-image, vs. D-MAD = differential, which needs a second trusted live-capture photo): we only have the two source photos that were morphed together, not an independent trusted photo of either person, so S-MAD actually fits our data.

The pipeline, step by step

1. Build the dataset -- bona fide class = all 116 original photos in outputs/pairs/; attack class = all 58 morphs in outputs/morphs/. Each item is tagged with its pair ID so two images from the same pair never end up on opposite sides of a train/test split.
2. Color-space expansion (to_color_channels) -- convert each image to YCbCr and HSV: 6 separate single-channel images. Different color channels expose different morphing artifacts.
3. Laplacian pyramid (laplacian_pyramid) -- for each of the 6 channels, build a 3-level pyramid isolating high-frequency detail at 3 scales. 6 channels x 3 levels = 18 'scale-space' sub-images per face.
4. Texture descriptors -- on every one of the 18 sub-images, compute LBP (blockwise texture histograms), HOG (gradient-direction histograms), and BSIF (a learned filter bank convolved + thresholded into a binary code, then histogrammed). Deviation from the paper: real BSIF uses filters pre-trained on 50,000 natural-image patches; we learn our own small filter bank via ICA from each fold's own bona fide training images instead. Each descriptor's 18 outputs concatenate into one long vector -> 3 feature vectors per face image.
5. Classification (mad_crc.py::CRCClassifier) -- for each of the 3 streams independently, a Collaborative Representation Classifier represents 'typical bona fide' and 'typical morph' feature vectors as two dictionaries (the stacked training vectors of each class). A probe's score = how much better it reconstructs as a morph than as a bona fide.
6. Fusion -- z-score-normalize each stream's scores against its own training distribution (so LBP/HOG/BSIF, with very different vector sizes, contribute comparably), then sum the 3 normalized scores into one final score.
7. Evaluation (train_mad.py) -- 58 pairs is tiny next to the paper's thousands, so we use 5-fold cross-validation split by pair ID (GroupKFold) instead of one fixed split, so every image gets exactly one held-out score. Metrics: APCER (attacks wrongly let through), BPCER (bona fide wrongly flagged), D-EER (where the two are equal) -- plotted as a DET curve.
8. Report (build_mad_report_pdf.py) -- reads outputs/mad/metrics.json and det_curve.png, writes submission_report_part2.pdf: chosen paper, method, our adaptations, dataset/protocol, results, limitations.

Headline result

Fused D-EER = 17.24% (LBP and HOG alone about 19% each; BSIF alone about 43%, near chance -- our self-learned filter substitution is weaker than the paper's real filter bank, but fusing it in still helps). The paper reports 5.6-6% D-EER, but on a dataset about 20x larger -- that gap is explained in the report rather than hidden.

Extra: a real neural network for the fusion step (mad_nn.py, train_mad_nn.py)

The paper's own fusion rule is a fixed sum -- no training, nothing to call 'weights' or 'loss'. Since this is a Neural Networks course, we added the simplest possible neural net on top: a single trainable neuron (logistic regression) that learns how much to weight the LBP/HOG/BSIF scores instead of just summing them: $z = w_{lbp} * lbp + w_{hog} * hog + w_{bsif} * bsif + b$, $p = \text{sigmoid}(z)$, trained by mini-batch stochastic gradient descent (batch size 8, 1000 iterations) on the binary cross-entropy loss. Its 3 inputs are the same out-of-fold CRC scores already computed by train_mad.py (already leakage-free), so this only needed one more pair-disjoint 80/20 train/test split (138 train / 36 test images) to fit and evaluate the neuron. A 2x2 dashboard (accuracy, loss, weight trajectory, bias trajectory -- train vs. test, per iteration) is saved to outputs/mad/nn_dashboard.png and embedded in the Part 2 report.

Results: training accuracy 92.03%, test accuracy 88.89%, final training/test cross-entropy loss 0.3505/0.3582. Learned weights: LBP +4.874, HOG +7.984, BSIF -1.580, bias -3.437. The network independently rediscovered, purely from data with no hand-tuning, that BSIF barely helps (its weight drifts negative over training) while LBP and HOG carry the real signal (both weights grow steadily positive) -- matching the per-stream D-EER numbers above.

How to run everything

```
# Part 1 (interactive app)
powershell -ExecutionPolicy Bypass -File .\run_app.ps1

# Part 1 report (already built, from pre-existing outputs/pairs & outputs/morphs)
python src/build_report_pdf.py

# Part 2: train + evaluate the detector, train the NN fusion layer, then build the report
python src/train_mad.py
python src/train_mad_nn.py
python src/build_mad_report_pdf.py
```

Quick cheat sheet for likely questions

Q: What morphing algorithm did you use?

A: Landmark-based Delaunay triangulation + piecewise affine warping + cross-dissolve (the classic Beier-Neely-style approach used in morphing-attack research, e.g. Ferrara et al. 2014).

Q: How do you get corresponding points on two different faces?

A: MediaPipe's FaceLandmarker gives the same 478-point topology for any face, so point i always means the same facial location on any image.

Q: Why 8 extra boundary points?

A: So the background and image edges are part of the triangulation too, not just the face -- otherwise everything outside the face would shear/distort.

Q: What dataset did you use?

A: LFW (Labeled Faces in the Wild), 58 random identity pairs, pre-generated (the generation script was later removed, only outputs kept).

Q: What's the difference between S-MAD and D-MAD, and which did you use?

A: S-MAD = decide from one image alone; D-MAD = compare against a second trusted live photo. We used S-MAD because we only have the two photos that were morphed together, no independent trusted reference.

Q: What features does your detector use?

A: LBP, HOG, and BSIF texture descriptors, computed over 6 color channels x 3 pyramid scales = 18 sub-images per face.

Q: What classifier?

A: Collaborative Representation Classifier (ridge-regression reconstruction residual), one per feature stream, fused by summing normalized scores.

Q: How did you evaluate it, and how good is it?

A: 5-fold cross-validation grouped by source pair (no data leakage), ISO/IEC 30107-3 metrics, D-EER = 17.24%.

Q: What did you have to change from the original paper?

A: BSIF filters learned from our own small training set via ICA instead of the paper's 50,000-patch natural-image bank; cross-validation instead of one fixed split, because our dataset is much smaller; z-score normalization added before score fusion.

Q: Where's the actual neural network -- weights, biases, cross-entropy loss, train/test accuracy?

A: The paper's own fusion step is just a fixed sum rule, so we added a single trainable neuron (logistic regression) on top of the 3 stream scores: $z = w_{lbp} * lbp + w_{hog} * hog + w_{bsif} * bsif + b$, sigmoid, trained by mini-batch SGD on binary cross-entropy (src/mad_nn.py, run via src/train_mad_nn.py), with a 2x2 diagnostics dashboard (accuracy, loss, weight trajectory, bias trajectory) saved to outputs/mad/nn_dashboard.png. Training accuracy 92.03%, test accuracy 88.89%, final train/test loss 0.3505/0.3582, weights (lbp +4.87, hog +7.98, bsif -1.58), bias -3.44. The negative BSIF weight is the network independently confirming BSIF is the weak stream, with no hand-tuning.